

## EXPERIMENT - 12

### 16 Bit division.

AIM:

To write an assembly language to implement 16 bit divided using 8086 processor.

ALGORITHM:

- 1) Read dividend (16) bit.
- 2) Read divisor.
- 3) count  $\leftarrow$  8.
- 4) left shift dividend.
- 5) Subtract divisor from upper 8 bits of dividend.
- 6) if  $cs = 1$ , go to 9.
- 7) Restore dividend.
- 8) Increment lower 8 bit of dividend.
- 9) count  $\leftarrow$  count - 1.
- 10) if count = 0, go to 5.
- 11) Store upper 8 bit dividend as remainder and lower 8 bit as quotient.
- 12) stop.

PROGRAM:

| 4NEMONICS       | EXPLANATION     |
|-----------------|-----------------|
| MOV AX, [1100H] | MOVE A to 1100. |

```

MOV BX, 1102H
DIV BX
MOV [1200H], AX
MOV [1202H], DX

HLT

```

Move B to 1102  
 Divide by base (B)  
 Move [1200] to A  
 Move [1202] to direction (D)  
 Halt.

Input

| Address | Data |
|---------|------|
| 1100    | 10   |
| 1102    | 10   |

Output :

| Address | Data |
|---------|------|
| 1200    | 1    |
| 1202    | 10   |

RESULT: 

Thus the program was executed  
 successfully using 8086 processor  
 simulator.

```
01 MOV AX, [1100H]
02 MOV BX, [1102H]
03 XOR DX, DX
04 DIV BX
05 MOV [1200H], AX
06 MOV [1202H], DX
07 HLT
08
09
10
11
```

Random Access Memory

0100:1100 update table list

|            |    |     |      |
|------------|----|-----|------|
| 0100:1100: | 10 | 016 |      |
| 0100:1101: | 00 | 000 | NULL |
| 0100:1102: | 10 | 016 |      |
| 0100:1103: | 00 | 000 | NULL |
| 0100:1104: | 00 | 000 | NULL |
| 0100:1105: | 00 | 000 | NULL |
| 0100:1106: | 00 | 000 | NULL |
| 0100:1107: | 00 | 000 | NULL |
| 0100:1108: | 00 | 000 | NULL |

emulator: noname.bin\_

file math debug view external virtual devices virtual drive help

Load reload step back single step run step delay ms: 0

| registers |       | 0100:0012          |  | 0100:0012        |  |
|-----------|-------|--------------------|--|------------------|--|
|           | H L   |                    |  |                  |  |
| AX        | 00 01 | 01000: A1 161 i    |  | MOV AX, [01100h] |  |
| BX        | 00 10 | 01001: 00 000 NULL |  | MOV BX, [01102h] |  |
| CX        | 00 00 | 01002: 11 017 4    |  | XOR DX, DX       |  |
| DX        | 00 00 | 01003: 8B 139 i    |  | DIV BX           |  |
| CS        | 0100  | 01004: 1E 030 4    |  | MOV [01200h], AX |  |
| IP        | 0012  | 01005: 02 002 0    |  | MOV [01202h], DX |  |
| SS        | 0100  | 01006: 11 017 4    |  | HLT              |  |
| SP        | FFFE  | 01007: 33 051 3    |  | NOP              |  |
| BP        | 0000  | 01008: D2 210 2    |  | NOP              |  |
| SI        | 0000  | 01009: F7 247 2    |  | NOP              |  |
| DI        | 0000  | 0100A: F3 243 1    |  | NOP              |  |
| DS        | 0100  | 0100B: A3 163 u    |  | NOP              |  |
| ES        | 0100  | 0100C: 00 000 NULL |  | NOP              |  |
|           |       | 0100D: 12 018 1    |  | NOP              |  |
|           |       | 0100E: 89 137 e    |  | NOP              |  |
|           |       | 0100F: 16 022 1    |  | NOP              |  |
|           |       | 01010: 02 002 0    |  | NOP              |  |
|           |       | 01011: 12 018 1    |  | NOP              |  |
|           |       | 01012: F4 244 f    |  | NOP              |  |
|           |       | 01013: 90 144 e    |  | NOP              |  |
|           |       | 01014: 90 144 e    |  | NOP              |  |
|           |       | 01015: 90 144 e    |  | ...              |  |

screen source reset aux vars debug stack flags

original source c...

```
01 MOV AX, [1100H]
02 MOV BX, [1102H]
03 XOR DX, DX
04 DIV BX
05 MOV [1200H], AX
06 MOV [1202H], DX
07 HLT
08
09
10
11
12
13
```